به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین پنجم

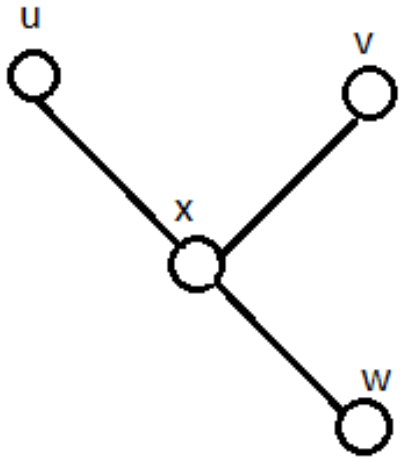
مدرس: مهدی صفرنژاد — انتشار: ۱۶ اردیبهشت ۹۹

گراف

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) چهارشنبه ۳۱ اردیبهشت ۹۹ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب سه‌شنبه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

سؤال ۱. [۱۰ نمره] فرض کنید یک گراف بدون جهت داریم که هر یال آن دارای دو وزن مثبت است. بار اولی که از یک یال عبور می‌کنیم باید به اندازه ی وزن بیشتر آن یال هزینه پرداخت کنیم و بارهای بعدی به اندازه‌ی وزن سبکتر هزینه پرداخت می‌کنیم. می‌خواهیم از راس u به راس v برویم و در مسیر از راس w عبور کنیم. الگوریتمی از مرتبه‌ی $O(n\log(n)+m)$ ارائه دهید که مسیر با کمترین وزن را پیدا کند که n و m تعداد رئوس و یال‌های گراف هستند.

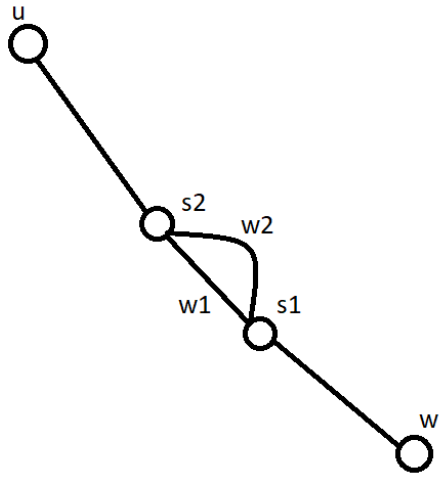
پاسخ: ادعا می‌کنیم جواب بهینه حتما به شکل زیر است:



در جواب بهینه مسیری که از u به w طی کرده‌ایم را p می‌نامیم. ادعا می‌کنیم بعد از رسیدن به w یخشی از مسیر p را بازمیگردیم و سپس بدون برگشت به رئوس مسیر p به راس v می‌رویم.

حال برای اثبات این ادعا، ثابت می‌کنیم بعد از عبور از راس w اولین جایی که از مسیر p خارج شویم دیگر به آن بر نمیگردیم. اولا اگر در مسیر p از راسی دوبار عبور نمی‌کنیم. در مسیر از w به v نیز این شرط برقرار است چراکه در غیر این صورت در مسیر یک دور داریم که می‌توان آن دور را بدون مشکل از مسیر حذف کرد و مسیر کوتاه‌تر می‌شود (یعنی هر مسیر راس تکراری در بین رئوس خود آن مسیر ندارد). حال برای اثبات ادعا ثابت می‌کنیم بعد از رسیدن به w اولین جایی که از مسیر p خارج شویم دیگر به آن برنمیگردیم.

برهان خلف: فرض کنید در جواب بهینه، مطابق شکل زیر بعد از رسیدن به راس w، از مسیر p تا راس s1 برمیگردیم و سپس از مسیر p خارج می‌شویم و دوباره در راس s2 به مسیر p برمیگردیم.



در این صورت هزینه‌ی رفتن از راس s1 به s2 در مسیر p را w1 و هزینه‌ی مسیر بین این دو راس که در خارج از p طی کرده‌ایم را w2 می‌نامیم. ادعا می‌کنیم اگر به‌جای مسیری که در خارج از p رفته‌ایم از داخل p از راس s1 به راس s2 می‌رفتیم هزینه‌ی مسیر کمتر می‌شد. چرا که با فرض بهینه بودن مسیری که طی کردیم داریم: $w۱ \leq w۲$ (در غیر اینصورت در مسیر از u به w اگر از بخش w2 می‌رفتیم هزینه کمتر می‌شد که با بهینه بودن مسیر در تناقض است). حال اگر مسیر بین s1 و s2 را دوباره در p برای دومین بار طی میکردیم، طبق فرض سوال باید هزینه‌ای کمتر از w1 پرداخت می‌کردیم چون به اندازه‌ی وزن کمتر یال‌ها اینبار هزینه می‌کنیم. از داخل p از s1 به s2 می‌رفتیم (به جای اینکه از pخارج شویم) هزینه‌ای کمتر از w1 پرداخت می‌کردیم که چون $w۱ \leq w۲$ پس هزینه‌ی مسیر جدید کمتر از مسیر قبلی می‌شود و با فرض بهینه بودن مسیری که طی کرده‌ایم در تناقض است. بنابراین فرض خلفی که کردیم به تناقض رسید و در نتیجه ادعای ما اثبات می‌شود.

حال برای اینکه مسیر بهینه را بیابیم روی راس x (که در شکل اول مشخص شده‌است) حالت‌بندی می‌کنیم و به ازای هر کدام از رئوس گراف بدست می‌آوریم اگر راس x ما (یعنی راسی که مسیر ما در آن به سه شاخه منشعب می‌شود) این راس باشد، هزینه‌ی پرداختی چقدر خواهد بود و به ازای تمام حالات کمترین جواب را بدست می‌آوریم.

حال فرض کنید برای راسی مثل y می‌خواهیم بدست بیاوریم اگر از u به w برویم و سپس به راس y در مسیر برگردیم و از y به v برویم هزینه چقدر خواهد بود. در این مسیر یال‌های از راس u تا y را با وزن بیشتر آن‌ها می‌رویم و بعد برای یال های بین y و w به اندازه‌ی جمع دو وزن کوچک و بزرگ یال آن‌ها هزینه می‌کنیم و برای یال‌های بین y و v هم به اندازه‌ی وزن بیشتر آن‌ها هزینه می‌کنیم. بنابر این یک بار روی یال‌های با وزن بیشتر از راس u الگوریتم dijkstra را اجرا می‌کنیم و به ازای هر راس مثل i، کمترین فاصله این راس تا راس u را dis1[i] می‌نامیم که با $o(nlogn + m)$ بدست می‌آید. همچنین یکبار این کار را برای راس v انجام می‌دهیم و کمترین فاصله هر راس تا راس v را با dis2[i] نمایش می‌دهیم (باز هم با $o(nlogn + m)$). یکبار هم وزن هر یال را جمع دو وزن آن قرار داده و روی گراف مذکور از راس w الگوریتم dijkstra را اجرا می‌کنیم و فاصله‌ها را با dis3[i] نمایش می‌دهیم $o(nlogn + m)$.

حال به ازای هر راس $d[i] = dis1[i] + dis2[i] + dis3[i]$ را محاسبه می‌کنیم (که d[i] یعنی اگر راس وسط ما، که از آن راس مسیر منشعب می‌شود، راس i باشد، کمترین هزینه‌ی مسیر چقدر خواهد بود) که با $o(n)$ قابل محاسبه است و کمترین d[i] جواب مساله است. پس در کل با $o(nlogn + m)$ جواب را بدست می‌آوریم.

سؤال ۲. [۱۰ نمره] روزی شخصی به صرافی می‌رود و با دیدن نرخ تبدیل ارزها ایده‌ای به ذهنش می‌رسد. او می‌بیند که می‌تواند با هر پوند ۱/۲ دلار، با هر دلار ۰/۹ یورو و با هر یورو ۰/۹۵ پوند بخرد. به این ترتیب او می‌تواند با تبدیل یک پوند به دلار، دلار به یورو و یورو به پوند صاحب پوند ۱/۰۲۶ = ۰/۹۵ * ۰/۹ * ۱/۲ پوند شود و ۰/۰۲۶ پوند سود کند. فرض کنید n واحد پول داریم و یک ماتریس T که T[i][j] یعنی با یک واحد از c[i] می‌توان T[i][j] واحد از c[j] خرید.

آ. [۷ نمره] یک الگوریتم از $O(n^۳)$ ارائه کنید که با ورودی گرفتن ماتریس T مشخص کند آیا دنباله‌ای از واحدهای پول مانند c[i[k]], ..., c[i[2]], c[i[1]] وجود دارد که تبدیل زنجیره‌ای از c[i[1]] به c[i[2]] و ...نهایتا از c[i[k]] به c[i[1]] باعث سود شود.

پاسخ: فرض کنید دنباله‌ای از واحدهای پول مثل c[1], c[2], ..., c[k] وجود دارد که تبدیل واحدها از c[1] به c[2] و c[2] به c[3] و ... و در نهایت از c[k] به c[1] که درنهایت منجر به سود شود. در این صورت داریم:

$$t[c[۱]][c[۲]] * t[c[۲]][c[۳]] * ... * t[c[k-۱]][c[k]] * t[c[k]][c[۱]] > ۱$$

$$\longrightarrow log(t[c[۱]][c[۲]] * t[c[۲]][c[۳]] * ... * t[c[k-۱]][c[k]] * t[c[k]][c[۱]]) > ۰$$

$$\longrightarrow log(t[c[۱]][c[۲]]) + log(t[c[۲]][c[۳]]) + ... + log(t[c[k]][c[۱]]) > ۰$$

$$\longrightarrow -log(t[c[۱]][c[۲]]) - log(t[c[۲]][c[۳]]) - ... - log(t[c[k]][c[۱]]) < ۰$$

حال یک گراف می‌سازیم به این شکل که متناظر با هر واحد پول یک راس در گراف قرار می‌دهیم و به ازای هر دو راسی مثل i و j، از راس i یالی با وزن $-log(t[i][j])$ به راس j می‌گذاریم که در $O(n^۳)$ قابل انجام است.
با توجه به محاسباتی که در بالا انجام دادیم پیداکردن دنباله‌ای از تبدیل‌ها که به سود منجر شود معادل است با پیداکردن دور منفی در گراف ساخته شده که با الگوریتم بلمن در $O(n^۳)$ انجام می‌گیرد.

ب. [۸ نمره] یک الگوریتم از $O(n^۳)$ ارائه کنید که در صورتی که دنباله‌ای با شرایط بالا وجود داشته باشد، یک دنباله‌ی دلخواه با آن شرایط را خروجی دهد.

پاسخ: par[i] را تعریف میکنیم در کوتاهترین مسیر از مبدا به راس i، فاصله‌ی این راس از روی چه راسی update شده است (یعنی هربار که یک یال مثل یال (i, j) را relax می‌کنیم، اگر فاصله‌ی راس j از روی راس i بدست آمد، مقدار par[j] را برابر i قرار می‌دهیم. درواقع par[i] می‌گوید در کوتاهترین مسیر از مبدا به این راس تا به الان،راس قبل از راس i چه راسی است. حال اگر بعد از پایان مراحل relax کردن راسی پیدا شد که با relax کردن مجدد فاصله‌اش تغییر کند (همان شرطی که برای داشتن دور منفی بررسی می‌کنیم)، یعنی در مسیر از مبدا به این راس یک دور منفی وجود دارد. بنابر این اگر از این راس شروع کنیم و در هر مرحله از راسی که در آن هستیم به par آن برویم، چون کوتاهترین مسیر از مبدا به این راس را طی می‌کنیم که دارای دور منفی است، پس به راسی تکراری خواهیم رسید که رئوس بین این راس تکراری همان دور منفی ما را تشکیل می‌دهند (چون در کوتاهترین فاصله قطعا دور مثبت نداریم و می‌توان با حذف یال از آن مسیر را کوتاه‌تر کرد پس دوری که پیدا می‌شود حتما منفی است).

سؤال ۳. [۱۰ نمره] یک گراف n راسی با یال‌های بدون جهت داریم. الگوریتمی با پیچیدگی زمانی $O(n + m)$ ارائه کنید و در آن یال های این گراف را طوری جهت‌دار کنید که کمترین تعداد مولفه‌ی قویا همبند تشکیل شود.

پاسخ: برای هر مولفه‌ی همبندی به طور جداگانه مساله را حل می‌کنیم چرا که واضح است رئوس دو مولفه‌ی جدا را نمی‌توان در یک مولفه قویا همبندی قرار داد. ادعا می‌کنیم اگر گراف همبند باشد کمینه تعداد مولفه‌های قویا همبند که به آن می‌توان رسید برابر است با $k + 1$ که k برابر است با تعداد یال‌های برشی گراف (یال برشی: یالی که با حذف آن تعداد مولفه‌های همبندی گراف افزایش می‌یابد). ابتدا ثابت می‌کنیم جواب حداقل برابر این تعداد است:
اگر یک یال برشی مثل یال (u, v) داشته باشیم نمی‌توان کاری کرد که دو سر آن در یک مولفه‌ی قویا همبند قرار بگیرند چرا که با حذف این یال برشی، دوسر آن در دو مولفه‌ی همبندی جداگانه قرار می‌گیرند پس امکان ندارد طوری جهت دهی کنیم که هم u به v مسیر داشته باشد و هم بالعکس و تنها مسیر بین این دو راس، یال بین آن‌هاست درحالیکه باید حداقل دو مسیر بین این دو راس باشد تا هر دو به هم مسیر جهت‌دار داشته باشند.
بنابراین یال‌های برشی در جواب بهینه درون یک مولفه‌ی قویا همبند نیستند (چون ثابت کردیم دو سر آن نمی‌توانند در یک مولفه قویا همبند قرار بگیرند) و بین دو مولفه‌ی قویا همبند مختلف هستند پس با حذف آن‌ها تغییری در تعداد مولفه‌های قویا همبند در جواب بهینه ایجاد نمی‌شود. بنابراین می‌توانیم یال‌های برشی گراف را حذف کنیم و جواب ثابت می‌ماند. اگر k یال برشی داشته باشیم بعد از حذف آن ها گراف به $k + 1$ مولفه‌ی همبندی تقسیم می‌شود که هرطور یال‌های آن را جهت‌دهی کنیم تعداد مولفه‌های قویا همبند حداقل $k + 1$ خواهد بود. بنابراین گراف اصلی هم که جوابش برابر جواب این گراف است حداقل به $k + 1$ مولفه‌ی قویا همبند تقسیم می‌شود و از این کمتر نمی‌شود.
حال اثبات می‌کنیم می‌توان طوری یال‌ها را جهت‌دهی کرد که تعداد مولفه‌های قویا همبند دقیقا برابر $k + 1$ باشد:
روی گراف از یک راس dfs می‌زنیم و یال‌های درخت را رو به پایین و back edge ها را رو به بالا جهت‌دهی می‌کنیم. این کار را با $O(n + m)$ می‌توان انجام داد. به استقرای ریاضی ثابت می‌کنیم اگر یال‌های گراف را به این شکل جهت‌دهی کنیم تعداد مولفه‌های قویا همبند تشکیل شده برابر است با $k + 1$.
لم: یال بین دو راس (v, p) برشی است اگر این یال در درخت dfs آمده باشد و اگر p پدر راس v در درخت dfs باشد، در زیردرخت راس v هیچ یال back edge به هیچ کدام از اجداد v وجود نداشته باشد. اثبات آن هم واضح است چرا که اگر یالی در درخت نیامده باشد، با حذف آن یال، گراف همچنان با یال‌های زیردرخت dfs همبند می‌ماند. اگر هم در درخت آمده باشد و شرط گفته شده را نداشته باشد، با حذف این یال دو راس v و p در دو مولفه همبندی جداگانه قرار می‌گیرند. برای اثبات این موضوع هم فرض خلف می‌کنیم که v بعد از حذف این یال همچنان به راسی خارج از زیردرختش مثل p راه داشته باشد. در این صورت مسیر از v به p را درنظر بگیرید. اولین جایی که به راسی خارج از زیر درخت v می‌رویم از یک راس داخل زیر درخت v به یک راس خارج از این زیردرخت رفته‌ایم درحالیکه فرض کردیم یالی از زیردرخت v به خارج آن وجود ندارد که تناقض دارد. پس لم اثبات شد.
فرض استقرا: اگر گراف همبند ما k یال برشی داشته باشد و یال های زیردرخت dfs آن را رو به پایین و سایر یال‌ها را رو به بالا جهت‌دهی کنیم تعداد مولفه‌های قویا همبند تشکیل شده برابر است با $k + 1$.
پایه: $k = 0$
در این حالت هیچ یال برشی‌ای در گراف نداریم. می‌خواهیم ثابت کنیم گراف قویا همبند خواهد بود. از راس ریشه که از آن dfs زده‌ایم که با یال‌های درخت (که رو به پایین جهت‌دهی شده‌اند) می‌توان به همه‌ی رئوس این مولفه رسید. حال ثابت می‌کنیم از هر راسی هم می‌توان به راس ریشه رسید. راس دلخواه u را در نظر بگیرید. از آنجاییکه گراف طبق فرض یال برشی ندارد و با توجه به

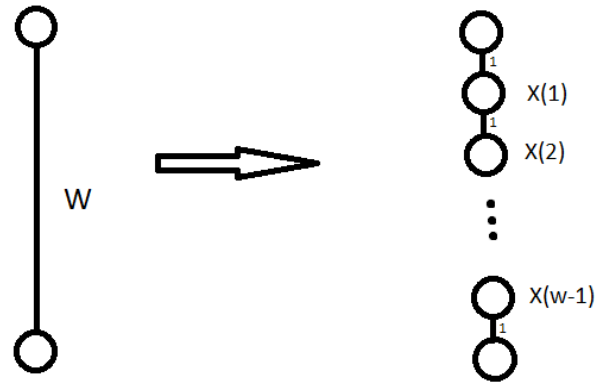
لم، حتما یک back edge از زیردرخت u به یکی از اجداد u وجود دارد. از u با یال‌های درخت به آن راس رفته (که همانطور که گفتیم رو به پایین جهت‌دهی شده‌اند) و سپس با back edge گفته شده به جد u می‌رویم (که جهت آن همانطور که گفتیم روبه بالا است). حال همین کار را برای جد u که به آن رسیده‌ایم انجام می‌دهیم و به جد راس جدید می‌رویم. در هر مرحله با تکرار این کار به یکی از اجداد راسی که در آن هستیم می‌رویم و فاصله‌مان هربار با ریشه کمتر می‌شود تا اینکه بالاخره به ریشه می‌رسیم. پس با جهت‌دهی گفته شده از هر راسی به راس ریشه می‌توان رسید. پس گراف ما قویا همبند است چراکه راس ریشه به تمام رئوس راه دارد و تمام رئوس هم به ریشه راه دارند.

حکم: اگر گراف همبند ما $k + 1$ یال برشی داشته‌باشد و با روش گفته‌شده یال‌های آن را جهت‌دهی کنیم، تعداد مولفه‌های قویا همبندی برابر است با $k + 2$.

چون $k+1 \geq 1$ پس حتما یال برشی داریم. بین یال‌های برشی یالی که بیشترین عمق را دارد درنظر بگیرید (عمق: فاصله از ریشه). فرض کنید این یال بین راس p و v باشد که راس p پدر راس v است. زیردرخت راس v را درنظر بگیرید. در این زیر درخت یال برشی نداریم چراکه اگر داشتیم عمق آن بیشتر از عمق یال (p, v) بود و با این فرض که (p, v) بین یال‌های برشی بیشترین عمق را دارد تناقض داشت. پس در زیر درخت راس v هیچ یال برشی نداریم و یال‌های داخل درخت آن روبه پایین و سایر یال‌ها رو به بالا جهت‌دهی شده اند. پس طبق فرض استقرا که برای حالت پایه گفتیم (گراف بدون یال برشی) کل این زیردرخت یک مولفه‌ی قویا همبند خواهد بود. حال این زیر درخت را از گراف اصلی حذف می‌کنیم. گراف باقیمانده گرافی همبند با k یال برشی است که طبق روش گفته شده جهت‌دهی شده است. پس بنابر فرض استقرا این گراف $k + 1$ مولفه‌ی قویا همبندی دارد که با یک مولفه‌ی قویا همبندی که حذف کردیم در کل $k + 2$ مولفه‌ی قویا همبندی داریم.

سؤال ۴. [۱۰ نمره] یک گراف با nراس و m یال داریم که وزن هر یال عددی بین ۱ تا L است. الگوریتمی با پیچیدگی زمانی $O(L(n + m))$ بیابید که کوتاهترین مسیر از راس ۱ به سایر رئوس را بیابد.

پاسخ: هر یال با وزن w را به w یال با طول یک تقسیم می‌کنیم به این شکل که $w - 1$ راس به گراف اضافه می‌کنیم و هر راس را به راس قبل و بعدش در روی یال وصل میکنیم که در شکل زیر نمایش داده شده‌است:



حال در گراف جدید هر یال به حداکثر L یال جدید تبدیل شده است و به ازای هر یال حداکثر $L - 1$ راس جدید به گراف اضافه شده است. پس در گراف جدید تعداد یال‌ها حداکثر برابر است با: $L*m$. همچنین تعداد رئوس گراف جدید حداکثر برابر است با: $n + m(L - 1)$.

حال در گراف جدید وزن هریال برابر با یک است و می‌توانیم از الگوریتم bfs برای پیداکردن کمترین فاصله از راس ۱ استفاده کنیم و از راس شماره ۱ bfs بزنیم که با $o(n + m(L - 1) + L * m)$ قابل محاسبه است که خود از $o(n + Lm)$ است.

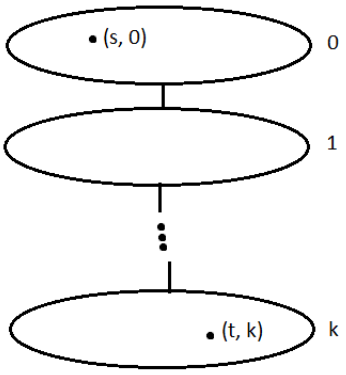
سؤال ۵. [۱۰ نمره] یگ گراف با nراس و m یال داریم. الگوریتمی با $O(n + m)$ ارائه کنید که بگوید گراف دوبخشی است یا خیر.
گراف دوبخشی: گرافی که بتوان رئوس آن را به دو دسته تقسیم کرد بطوریکه بین رئوس درون یک دسته یالی وجود نداشته باشد.

پاسخ: مساله معادل رنگ کردن راس‌های گراف با دو رنگ است به طوری که بین رئوس همرنگ یالی نباشد (رئوس هر رنگ را یک بخش از گراف دو بخشی در نظر می‌گیریم). برای این منظور، هر مولفه‌ی همبندی را بطور جداگانه بررسی می‌کنیم. اگر بتوان رئوس هر مولفه را به شکل گفته شده رنگ کرد (یعنی اگر هر مولفه از گراف دوبخشی باشد)، در این صورت کل گراف را هم می‌توان رنگ کرد چون رئوس هر مولفه به دو بخش تقسیم شده اند و بین رئوس دو مولفه‌ی همبندی جدا هم یالی نیست (وگرنه در یک مولفه‌ی همبندی بودند). اگر هم مولفه‌ای در گراف وجود داشته باشد که دوبخشی نباشد در این صورت کل گراف هم دوبخشی نیست. برهان خلف: فرض کنید مولفه‌ای دوبخشی نباشد اما کل گراف دوبخشی باشد: در اینصورت میتوان رئوس کل گراف را با دو رنگ رنگ‌آمیزی کرد که بین رئوس یک رنگ یالی نباشد و در نتیجه در همان رنگ‌آمیزی گراف، رئوس مولفه‌ی مدنظر هم به دو رنگ تقسیم می‌شوند که بین رئوسشان یالی نیست پس مولفه‌ی مدنظر هم دوبخشی بوده که با فرض اولیه ما (که دوبخشی نبودن این مولفه است) تناقض دارد.
حال برای چک کردن دوبخشی بودن هر مولفه، از یکی از رئوس آن مولفه dfs می‌زنیم. به راسی که از آن dfs زده‌ایم رنگ ۱ را نسبت می‌دهیم (بنابه تقارن فرقی نمی‌کند به راس اول کدام رنگ را نسبت دهیم و می‌توان همه رنگ های را برعکس کرد تا در جواب رنگ این راس همان رنگ مدنظر ما باشد). حال به فرزندان این راس در زیردرخت dfs مجبوریم رنگ ۲ را نسبت بدهیم تا دو سر یال‌ها همرنگ نباشد. به همین شکل به هر راس در درخت dfs مجبوریم رنگ مخالف با رنگ پدرش را نسبت بدهیم (یعنی از ریشه شروع می‌کنیم و به هر راس که در dfs می‌رسیم رنگ مخالف با رنگ پدرش را نسبت می‌دهیم چون اگر رنگ پدرش مشخص باشد مجبوریم این رنگ را به این راس نسبت دهیم تا بین رئوس همرنگ یال نباشد) که این کار با $o(n + m)$ قابل انجام است. پس اگر رنگ ریشه را مشخص کنیم، رنگ تمام رئوسی که در dfs به آن‌ها می‌رسیم به طور اجباری مشخص می‌شود و چون در dfs به تمام رئوس مولفه می‌رسیم، پس مشخص می‌شود اگر گراف بخواهد دوبخشی باشد رنگ هر راس چه خواهد بود. بعد از آن که رنگ ها را نسبت دادیم برای هر یال بررسی می‌کنیم آیا دوسر آن همرنگ هستند یا خیر (با $o(m)$) و اگر یالی با دوسر همرنگ پیدا شود گراف دوبخشی نیست (چرا که رنگ‌های رئوس به طور یکتا مشخص شده‌اند) و اگر هم چنین یالی پیدا نشد گراف دوبخشی بوده و با رنگ‌آمیزی ارائه شده آن‌ها را به دوبخش تقسیم کرده‌ایم. پس با $o(n + m)$ جواب را بدست می‌آوریم.

سؤال ۶. [۹ نمره] یک گراف وزندار و جهتدار با n راس و m یال داریم. وزن‌ها می‌توانند منفی هم باشند. s و t دو راس از گراف هستند و k عددی طبیعی است.
الگوریتمی از مرتبه زمانی $O(k(n + m))$ ارائه کنید که بین تمام مسیرهای از s به t که دقیقا k یال دارند، طول کوتاه‌ترین مسیر را پیدا کند.
راهنمایی : یک گراف جدید تشکیل دهید و به ازای هر راس در گراف اصلی مثل v، k+1 راس به شکل (v, i) درنظر بگیرید و یال‌ها را طوری قرار دهید که فاصله‌ی (v, i) از (s, 0) برابر با i باشد.

پاسخ: از روی گراف اصلی یک گراف جدید می‌سازیم به این شکل که از رئوس این گراف $k + 1$ بار کپی می‌گیریم بطوریکه به ازای هر راس مثل v، $k + 1$ راس در گراف جدید با نام‌های (v, 0), (v, 1), ..., (v, k) خواهیم داشت و $k + 1$ لایه با شماره‌های

• تا k از گراف اصلی تشکیل شود. به ازای هریال در گراف اصلی مثل یال u به v، در گراف جدید k یال قرار می‌دهیم که یال iام از راس (u, i - 1) به راس (v, i) خواهد بود و وزن آن همان وزن یال u به v است (یعنی از راس متناظر با u در هر لایه به راس متناظر با v در لایه‌ی پایین‌تر یال می‌گذاریم). گراف جدید دارای n(k + 1) راس و mk یال خواهد بود.



حال ادعا می‌کنیم هر گشت با طول k در گراف اصلی از راس s به راس t، معادل یک مسیر از راس (s, 0) در گراف جدید به راس (t, k) خواهد بود. به این شکل که از راس (s, 0) شروع می‌کنیم و اگر یال iام گشت از u به vبود، در گراف جدید از راس (u, i - 1) به راس (v, i) می‌رویم. بنابراین بعد از طی کردن یک گشت به طول k در گراف اصلی از s به t، در گراف جدید یک مسیر از (s, 0) به (t, k) خواهیم داشت. برعکس آن هم برقرار است یعنی هر مسیر از (s, 0) به (t, k) در گراف جدید معادل یک گشت به طول k از راس s به راس t در گراف اصلی خواهد بود به این شکل که اگر یالی مثل (x, i) به (y, i + 1) را در گراف جدید طی کردیم در گراف اصلی از راس x به راس y می‌رویم و بنابراین بعد از طی کردن k راس t می‌رسیم.

بنابراین پیدا کردن کوتاه‌ترین گشت با k یال از راس s به راس t در گراف اصلی معادل است با پیدا کردن کوتاه ترین مسیر از راس (s, 0) به راس (t, k) در گراف اصلی. حال تعریف می‌کنیم: dis(v, i) برابر است با کوتاهترین مسیر از راس (s, 0) به راس (v, i) در گراف جدید. بنابراین dis(v, i) = min(dis(u, i - 1) + w(u, v) به ازای همه‌ی رئوس (u, i - 1) که به (v, i) یال دارند و w(u, v) برابر وزن یال از u به v است چراکه هر راس از رئوس در لایه‌ی بالاتر update می‌شود و فرض می‌کنیم در هرمرحله فاصله رئوس لایه‌های قبلی محاسبه شده‌است و بنابراین فاصله‌ی رئوس این لایه هم به درستی محاسبه می‌شود و برای بدست آوردن مقدار dis برای هر راس، به اندازه‌ی تعداد یال‌های ورودی آن راس زمان می‌برد که در مجموع برابر با کل تعداد یال‌ها در گراف جدید می‌شود و از o(mk + nk) هزینه می‌کنیم و جواب برابر است با dis(t, k).

(برای راه مشابه می‌توان گفت گراف ساخته شده DAG است و دور ندارد چراکه یال ها از هر لایه به لایه پایین‌تر است و بنابراین می‌توان یک ترتیب TOPOLOGICAL از رئوس با O(k(n + m)) پیدا کرد که یال‌ها از هر راس به رئوس بعدی است و سپس براساس این ترتیب فاصله هرراس را از روی فاصله‌ی رئوسی که به آن یال دارند بدست می‌آوریم که چون ترتیب TOPOLOGICAL است فاصله رئوس همسایه قبلا محاسبه شده اند و با o(k(n + m)) بدست می‌آید.)

سؤال ۷. شما n نقطه روی صفحه‌ای کشیده‌اید و بازی‌ای را با شنگدباو شروع می‌کنید. او هر سری به شما دو عدد u, v می‌گوید و شما باید یال بین u و v را روی کاغذ بکشید. اگر گرافی که با یال‌های کنونی‌ای که کشیده‌اید دوبخشی نبود باید بگویید «بینگو» و بازی تمام می‌شود.

ورودی:
در خط اول به شما دو عدد n, m داده میشود در خط i ام i امین یالی که قرار است شما اضافه کنید قرار دارد و به صورت دو عدد u, v آمده است که به این معناست که شما باید یالی بین u, v اضافه کنید

$$۱ \leq n, m \leq ۱۰^۵$$

خروجی:
تنها یک عدد i را چاپ کنید که این به این معناست که پس از افزودن i امین یال دیگر گراف دوبخشی نیست و شما باید بگویید «بینگو».

محدودیت‌ها:
$۱ \leq n, q \leq ۱۰^۶$
$۰ \leq |s[i]| \leq ۱۰^۹$
$۱ \leq a, b \leq n$

ورودی و خروجی نمونه:

stdin	stdout
4 6 1 2 3 4 3 2 1 4 1 3 2 4	5

پاسخ: می‌دانیم از یک جایی به بعد دارای دور فرد شده و دوبخشی نمی‌شود. ما باید اولین جاییکه دور فرد ایجاد شده را پیدا کنیم. می‌توانیم جست‌وجوی دودویی روی این مکان بزنیم و به ازای هر اندیس مورد جست‌وجو تمامی یال‌ها تا قبل از آن اندیس را در نظر می‌گیریم. با استفاده از یک dfs می‌توان فهمید که آیا گراف دارای دور فرد هست یا نه. به اینصورت که رنگ هر کس را یا صفر یا یک قرار می‌دهیم و در dfs هر رأس را رنگ مخالف پدرش می‌گذاریم. می‌دانیم درخت dfs مجبور می‌کند گراف اینطوری رنگ شود پس اگر رنگ‌آمیزی ارائه شده توسط جست‌وجوی عمق اول ما مطلوب نبود یعنی یالی داشت که دو سرش رنگ برابر داشتند یعنی دوری فرد داریم. چک کردن چنین چیزی معادل چک کردن هر یال است در کل الگوریتم $\mathcal{O}(nlog_۲n)$ طول می‌کشد.

سؤال ۸. شنگدباو باید در هر لحظه حرکت کند و مجاز نیست سر جای خود بایستد. همچنین یال‌ها یکطرشنگدباو در گرافی n رأسی و m یالی وزن‌دار و جهت‌دار گیر افتاده‌است! او در ابتدا در رأس s قرار دارد و برای خروج باید به رأس t برود که در آن دری به خارج از گراف وجود دارد. نکته‌ای که وجود دارد این است که در مورد نظر در ابتدا باز است اما با گذشتن هر ثانیه در تغییر وضعیت می‌دهد یعنی در ثانیه اول باز است و در ثانیه دوم بسته و در ثانیه سوم باز و ... .

فه هستند لذا روی هر یال صرفاً به یک جهت می‌تواند حرکت کند. وزن نوشته‌شده روی هر یال برابر طول آن یال است یعنی اگر وزن یالی برابر c باشد، شنگدباو برای رسیدن از ابتدا به انتهای آن باید c ثانیه بدود!

به او کمک کنید تا کمترین زمانی که طول می‌کشد تا او از شر این گراف خلاص شود را بیابد.

ورودی:
در خط اول ورودی دو عدد n و m به ترتیب داده‌می‌شود که به معنای تعداد رأس‌ها و یال‌های گراف مورد نظر است. سپس در خط بعدی شماره دو رأس s و t به ترتیب داده می‌شود.

در m خط بعدی در هر خط سه عدد u و v و c می‌آیند که بیانگر رأس ابتدایی و انتهایی و وزن یال متناظر است.

$$۱ \leq n, m \leq ۱۰^۵$$

وزن یال‌ها نیز حداکثر $۱۰^۹$ می‌باشد.

خروجی:
اگر شنگدباو نمی‌توانست از گراف خارج شود -۱ خروجی دهید و در غیر این‌صورت کمترین تعداد ثانیه‌هاییکه طول می‌کشد تا او از گراف خارج شود را خروجی دهید.

ورودی و خروجی نمونه:

stdin	stdout
5 2 1 2 1 2 1 2 1 1	-1

پاسخ: می‌توانیم گرافی با $۲n$ رأس بسازیم و به ازای هر رأس u در گراف اصلی دو رأس u_{odd} و u_{even} بسازیم که وقتی به هر کدام از این رأس‌ها می‌رسیم اگر در u_{odd} باشیم یعنی زوجیت مسیر طی شده فرد است و اگر در u_{even} باشیم یعنی زوج است. حالا یال‌ها را به گونه‌ای می‌گذاریم که چنین شرطی را ارضا کند. فرض کنید یک یال با وزن w از u به v داریم آنوقت اگر w فرد بود دو تا یال زیر را اضافه می‌کنیم:

$$u_{odd} \to u_{even}$$

$$u_{even} \to u_{odd}$$

وزن هر دوی این‌یال‌ها برابر همان وزن یال uv یعنی $w(u \to v)$ است. همچنین اگر w زوج بود دو یال زیر را اضافه می‌کنیم:

$$u_{odd} \to u_{odd}$$

$$u_{even} \to u_{even}$$

که وزن آن همان $w(u \to v)$ است. می‌توان دید کوتاه‌ترین مسیر از s_{even} به t_{even} دقیقاً معادل خواسته مسئله در این گراف جدید است. می‌دانیم تعداد رئوس و یال‌ها در این گراف به ترتیب $۲n$ و $۲m$ است بنابراین با اجرای الگوریتم دایکسترا می‌توانیم با $\mathcal{O}((۲m + ۲n)log_۲(۲n))$ به جواب مسئله برسیم.

سؤال ۹. شنگدباو n تا قفس دارد که در هر کدام موشی قرار دارد. تعدادی جفت از این قفس‌ها به صورت یکطرفه به هم متصل اند و موشی که در قفس اول باشد می‌تواند به قفس دوم در آن جفت برود.

رفتار موش‌های شنگدباو عجیب است! موش‌های او اگر در قفسی باشند که به i تا قفس دیگر راه‌داشته باشد به احتمال $\frac{۱}{i}$ به یکی از قفس‌های دیگر می‌روند. فرض کنید شنگدباو موش‌ها را در این قفس‌ها با راه‌های بینشان رها کرده‌است و N روز بعد می‌خواهد برگردد و موش‌ها را جمع‌آوری کند. می‌دانیم N عدد بزرگی است و می توانید فرض کنید $N \to \infty$.

سؤال اینجاست که پس از این تعداد روز شنگدباو حداقل چندتا قفس را باید بردارد تا تضمین شود به احتمال حدوداً «۱» همه موش‌ها را جمع آوری کرده‌است.

راهنمایی: از مؤلفه‌های قویاً همبند گراف قفس‌ها و راه‌ها استفاده کنید و ببینید پس از تعداد زیادی روز موش‌ها در کدام مؤلفه‌ها ممکن است جمع شوند.

ورودی:
در خط اول ورودی به‌شما n و m داده می‌شود که بیانگر تعداد قفس‌ها و تعداد راه‌های یکطرفه بینشان است. سپس در m خط بعدی دو عدد u, v داده‌می‌شود که به معنای وجود مسیری از قفس u به قفس v است.

$$۱ \leq n, m \leq ۱۰^۵$$

تضمین می‌شود راهی از یک قفس به خودش نیست.

تضمین می‌شود دو تا جفت برابر در m خط ورودی داده نمی‌شود. بعبارتی بین هر دوتایی (u, v) حداکثر یک‌راه وجود دارد.

خروجی:
تنها یک عدد خروجی دهید و آن‌هم برابر تعداد کمینه قفس‌هایی باشد که با جمع کردنشان شنگدباو همه موش‌ها را جمع‌آوری می‌کند.

ورودی و خروجی نمونه:

stdin	stdout
5 5 1 2 1 3 2 4 3 4 2 5	2

پاسخ: اگر گراف فاقد دور باشد دارای تعدادی رأس sink یعنی بدون خروجی است. به‌سادگی می‌توان دید جواب مسئله برابر تعداد این رئوس است یعنی پس از n مرحله موش‌ها نهایتاً در یکی از این رئوس می‌روند. برای اثبات چنین چیزی ترتیب توپولوژیک گراف را در نظر بگیرید می‌توان دید پس از مرحله i ام اگر رأس iام یک sink نباشد خالی می‌شود و دیگر پر نمی‌شود. در نهایت کافیست روی رئوس پیمایش کنیم و تعداد رئوس بدون خروجی را بشماریم.

اما به ازای حالت کلی باید مؤلفه‌های قویا همبند را محاسبه کرد و به جای تک رأس یکسری مؤلفه‌ها در sink قرار دارند. می‌توان اثبات کرد پس از بی‌نهایت مرحله احتمال حضور موش در مؤلفه‌های غیر sink برابر صفر است. شهود آن این است که هر سری به احتمال کمی ممکن است موشی از این مؤلفه خارج شود و به مؤلفه بعدی در این ترتیب توپولوژیک برود، ممکن است این احتمال کم باشد اما به ازای بی‌نهایت مرحله می‌توان دید احتمال خضور موش‌ها در اینجا به صفر میل می‌کند.(برای اثبات دقیق می‌توانید در مورد زنجیرهای مارکف مطالعه کنید!).

حالا برای حل مسئله باید با استفاده از الگوریتم پیدا کردن مؤلفه‌های قویاً همبند مؤلفه‌های گراف را پیدا کنید و ببینید چندتا مؤلفه sink دارد، هر رأسی داخل این مؤلفه‌ها احتمال حضور موش را دارد!

موفق باشید